K-ORCH: an orchestrator for distributed multikernel instances

Eduardo Kaluf[1], **Carlos Alberto Maziero**[2]

[1] Departamento de Informática – Universidade Federal do Paraná (UFPR)
Curitiba – PR – Brazil

eduardo@kaluf.com.br, maziero@inf.ufpr.br

Abstract. *The multikernel architecture runs multiple independent kernel instances, each on a dedicated partition of cores and memory of the same processor, being a middle ground between containers and ~~VMs and~~ virtual machines, offering predictable performance, kernels with specialized configurations, and performance isolation between instances, without CPU and memory virtualization overhead. Current implementations of this architecture ~~[Wang 2025b]~~ manage these instances only within a single machine, and no tools exist to orchestrate them in distributed environments. This work proposes K-ORCH, an orchestrator that provisions, places, and monitors multikernel instances across multiple nodes, based on a desired-state model, agents responsible for partitioning cores and memory on demand, and placement policies driven by the available physical resources. K-ORCH will be experimentally evaluated through a quantitative analysis of provisioning time, control-plane overhead, density, and performance interference between instances, compared to orchestrated microVMs, in addition to comparing the orchestrator's own static and dynamic partitioning models. The expected outcome is to demonstrate the feasibility of distributed multikernel orchestration and to highlight its costs, advantages, and limits relative to current alternatives.*

1. Introduction

The number of cores per processor grows incessantly, and processors already exceed a hundred cores on a single chip [AMD 2024]. Monolithic operating systems, designed around data and control structures shared by all cores, have kept pace with this growth at the cost of increasingly complex mechanisms, such as fine-grained locks, per-core data structures, and CPU affinity and isolation policies [Boyd-Wickizer et al. 2010]. These solutions increase complexity and the kernel surface, and make state sharing between cores a bottleneck for performance and scalability [Boyd-Wickizer et al. 2008]. In this context, the multikernel architecture emerges, proposing to treat the machine as a network of independent cores, with replicated state and explicit message-based communication, instead of a single kernel with shared state, bringing the kernel itself closer to the idea of a distributed system [Baumann et al. 2009, Wentzlaff and Agarwal 2009].

Since then, two ways of logically subdividing the cores have been widely explored. The first restructures the ~~OS~~ operating system (OS) entirely, as in Barrelfish, which runs one kernel per core [Baumann et al. 2009]. The second line, adopted by this work, partitions the physical machine among multiple independent kernel instances, each

with its own cores and memory region, with the possibility of having different configurations and fine-tuning. This line emerged with high-performance computing (HPC) in mind, with systems that run lightweight kernels alongside Linux [Shimosawa et al. 2014, Park et al. 2012, Gerofi et al. 2016a], and was recently proposed for Linux itself, which now loads independent instances on reserved cores and memory, with run-time reallocation [Wang 2025b, Wang 2025a]. In this model, a host kernel partitions the resources and starts the instances, which communicate through explicit message passing and shared memory regions~~, in .~~ In addition, they access the rest of the hardware, in general, through virtual devices provided by the host kernel. In this work, *multikernel instance* refers to this set of objects: dedicated cores, its own memory region, an application image, and its own kernel configuration.

Furthermore, the multikernel architecture occupies a unique point between the two forms of resource subdivision that are widely used today. Containers are executable units of code that package an application and its dependencies, using mechanisms of the OS itself, such as *namespaces* and *cgroups*, to divide resources and limit them. Containers are lightweight, but they share the host kernel, which widens the attack surface between applications and leaves them subject to interference from neighbors (noisy neighbors). On the other hand, ~~VMs~~ virtual machines (VMs) run their own kernel on top of a hypervisor and offer stronger isolation, at the cost of an overhead from guest-host communication, I/O, and memory access, that is, they have higher resource consumption per instance. Thus, a multikernel instance comes in as a third way. It also runs its own kernel, but directly on the physical cores, without the CPU and memory virtualization layer of VMs, which favors predictable performance and allows the kernel to be specialized for specific workloads. On the other hand, allocation has whole-core granularity, and security isolation between instances depends on the hardware mechanisms of the machine.

The large-scale adoption of containers and VMs is strongly tied to the creation of orchestrators that manage these units, with emphasis on the line that went from Borg to Kubernetes [Burns et al. 2016]. Existing tools for multikernel, on the other hand, such as IHK [Shimosawa et al. 2014] and kerf [Multikernel Technologies 2025], offer mechanisms to create, configure, monitor, and destroy instances, similar to container tools such as *docker compose*, but they operate on a single machine, without defining policies about where and when these instances should be created. Without a robust orchestration layer, the architecture tends to remain restricted to single-machine experiments, far from distributed production environments such as the cloud, clusters, fog computing, and edge computing.

Therefore, this work proposes K-ORCH, an orchestrator for multikernel instances on nodes of a local network, under a single administrative domain, capable of meeting the specific needs of this emerging architecture. K-ORCH receives requests for instances described with the desired state, chooses a target node based on available cores and memory, delegates to an agent on each node the partitioning of resources and the initialization of the instance, and also monitors the instances in order to detect and recover from failures. The proposal will be evaluated in terms of provisioning time, control plane cost, density, and performance interference between instances, compared with orchestrated microVMs.

The remainder of this document is organized as follows. Section 2 presents the justification for the work along with the research question and hypothesis, the general

objectives are in Section 3 and the specific ones in Section 3.1. Section 4 discusses related work, and Section 5 describes the methodology. Finally, Section 6 presents the expected results, and Section 7, the two-year schedule for carrying out the work.

2. Justification and Motivation

The trade-off between isolation and performance described in Section 1 has concrete consequences in multi-tenant environments. Container escape vulnerabilities, such as CVE-2019-5736 and CVE-2024-21626 [NIST 2019, NIST 2024], make evident the risk of sharing the same kernel among workloads from different customers. In addition, interference between workloads leads providers to keep their servers underutilized, in order to preserve the latency of critical services [Lo et al. 2015]. Industry and academia ~~seek~~ have pursued intermediate solutions to ~~deal with these problems~~this trade-off, such as Firecracker [Agache et al. 2020], developed to run functions and containers from different customers in microVMs, and Kata Containers [OpenInfra Foundation 2026], which runs each pod (basic unit of Kubernetes) in a lightweight virtual machine. These efforts indicate that there is demand for alternatives to the two traditional models~~, where the multikernel architecture has a lot of room to establish itself as a third option~~. This work focuses on the performance side of this trade-off, namely performance isolation and kernel specialization. Security isolation between multikernel instances depends on the hardware protection mechanisms used by each implementation, and it will not be part of the work evaluation.

Multikernel is a ~~huge~~ significant advantage for large-scale applications in which specific OS configurations directly influence the performance of the instances being run, such as databases and monitors. This makes it unnecessary to dedicate an entire machine specifically to the applications, something that is not possible with containers due to sharing of the host kernel, avoiding costs and giving greater speed to the computation existing in these systems. In HPC environments, the use of lightweight kernels alongside Linux has already proven to be a viable alternative in massive production systems [Gerofi et al. 2016b]. In edge computing, where hardware is limited and latency must be predictable, execution without the CPU and memory virtualization layer is also a great advantage. Both are scenarios in which the use of multikernel instances, with specific configurations, would benefit providers.

The proposal for multikernel support in Linux [Wang 2025b] brings the architecture closer to the most widely used kernel on servers, but without tools that allow it to be expanded beyond a single machine. Just as the adoption of containers was extremely influenced by orchestrators, the adoption of multikernel in distributed environments depends on an equivalent layer. More than that, from a scientific point of view, characterizing the costs and limits of this orchestration is a contribution in itself, regardless of how the architecture behaves on the proposed criteria.

An alternative to building a dedicated orchestrator would be to integrate multikernel instances into Kubernetes through an API compatible with its interfaces, similar to what Kata Containers does with its lightweight VMs. This work chooses to explore a dedicated orchestrator mainly because placement depends on physical resources, such as free cores and reserved memory. Therefore, the intention is to investigate partitioning without being tied to an existing platform; in addition, an independent prototype makes

it possible to measure the cost of orchestration in isolation, with less interference and greater accuracy.

Therefore, the central question that serves as the basis of this work is: what mechanisms are needed to provision, place, and monitor multikernel instances across multiple nodes, and what is the cost of this management relative to the orchestration of microVMs?

The hypothesis of this work is that it is possible to orchestrate multikernel instances across multiple nodes such that:

1. The cost introduced by the orchestrator is a small fraction of the total provisioning time.
2. Instances exhibit less performance interference among themselves than microVMs under the same workloads and equivalent orchestration.
3. Dynamic resource partitioning increases node utilization compared to static partitioning.

3. Objectives

The general objective of this work is to design, implement, and evaluate K-ORCH, which must be capable of provisioning, placing, and monitoring multikernel instances across multiple nodes.

3.1. Specific objectives

- Survey the state of the art: identify the mechanisms of current orchestrators and of multikernel management tools, connecting the two areas, in order to determine which mechanisms can be reused and which need to be created or adapted specifically for K-ORCH.
- Define the orchestrable unit: characterize the multikernel implementation that will be used, including how cores and memory are reserved and reallocated, and formalize what makes up an instance and the desired-state model that the orchestrator must maintain.
- Design the architecture: define the operation of the control plane and its interaction with the data plane, design the agent responsible for instantiating the multikernel units from the host kernel of each node, and define the communication model between the agents and the orchestrator.
- Implement the lifecycle and placement of instances: implement the creation, initialization, stopping, and removal of instances and the placement mechanism, responsible for choosing the node that will serve each request based on free cores and available memory.
- Develop the resource partitioning model: implement a hybrid model that allows choosing between reserving memory and CPU regions at the initialization of each node or partitioning dynamically on each request.
- Implement instance monitoring: collect metrics on the state and resource usage of each node, detect failures and loss of communication with the agent, using monitoring to recover instances with errors, relocating or restarting them as necessary.
- Experimentally evaluate the orchestrator: measure provisioning time, the overhead of the agent and the control plane, instance density per node, ~~and~~ performance interference between instances, failure detection and recovery times,

comparing the results with ~~orchestrated microVMs~~ microVMs orchestrated under equivalent conditions, and compare static ~~and~~ against dynamic partitioning models.

K-ORCH will be designed and evaluated for nodes connected on the same local network and under a single administrative domain. In this scenario, it is assumed that the nodes are administered by the same entity and that the network offers low latency. The extension to multiple local networks under the same administrative domain, such as in a datacenter, is considered a natural evolution of the work. The orchestration of nodes distributed over the Internet, under multiple administrative domains, is outside the scope of this work.

4. Related Work

This section presents related work, showing the evolution of the multikernel architecture and connecting other works to the focus of this proposal: provisioning, placing, and monitoring multikernel instances across different nodes. Section 4.1 presents the evolution of architectures with multiple kernels, from systems for shared-memory multiprocessors to the multikernels used in HPC environments and the recent proposal for Linux. Section 4.2 discusses how these systems decide on resource allocation, how they monitor them, and how they ensure the isolation and availability of instances. Section 4.3 addresses current orchestrators and multikernel management tools.

4.1. Architectures with multiple kernels

The idea of structuring the OS as a set of independent kernels emerged a few decades ago. Hive [Chapin et al. 1995] organizes the OS as an internal distributed system of kernels, called cells, where each one is responsible for a fixed set of nodes and operates as an independent OS, while they cooperate to offer applications a single, transparent system image. Communication between cells occurs through *remote procedure calls* (RPC) and by directly reading the memory of other cells.

Instead of rewriting the OS entirely, Disco [Bugnion et al. 1997] inserts a small virtual machine monitor that runs several copies of an OS on the same machine, communicating through common distributed protocols, such as NFS and TCP/IP, which makes it possible to run specialized operating systems side by side with commercial systems. Cellular Disco [Govil et al. 2000] extends the proposal by dividing the monitor itself into smaller parts, in order to contain hardware faults, turning the multiprocessor into what would be a virtual cluster.

Corey [Boyd-Wickizer et al. 2008], by contrast, starts from the principle that applications should control sharing. By default, each kernel data structure is used by a single core, and abstractions such as *address ranges*, *kernel cores*, and *shares* allow the application to explicitly indicate what should be shared. Although it runs a single kernel, Corey was an important foundation for later works, such as Barrelfish. FOS [Wentzlaff and Agarwal 2009], in turn, proposes factoring OS services into fleets of servers spatially distributed across the cores, with communication exclusively through messages, motivated by the finding that the Linux physical page allocator did not scale beyond eight cores. Message-based communication became established in subsequent proposals, including the multikernel implementation based on Linux [Wang 2025b].

The modern formulation of the concept is presented by Baumann et al. [Baumann et al. 2009], who propose treating the machine as a network of independent cores. The resulting system, Barrelfish, runs one kernel per core and follows three principles: communication between cores is always explicit, the structure is hardware-neutral, and state is replicated instead of shared, being kept consistent by monitor processes.

More recently, the proposal for supporting multiple Linux instances on the same machine [Wang 2025b] loads the instances via *kexec*, assigning each one cores and memory that can be reallocated at run time through a *device tree* and the *hotplug* mechanism [Wang 2025a]. This is the implementation of the architecture planned for K-ORCH.

4.2. Load balancing, observability, isolation, and fundamental concepts

Deciding where to run each workload is a problem that has long been studied in distributed systems. Casavant and Kuhl [Casavant and Kuhl 1988] propose a hierarchical scheduling taxonomy that distinguishes static from dynamic approaches and classifies the latter according to the distribution of responsibility for decisions and the cooperation between nodes. Shivaratri et al. [Shivaratri et al. 1992] decompose load distribution algorithms into four policies: transfer (when to move a task), selection (which task to move), location (where to move it), and information (when to collect the state of the nodes). Mitzenmacher [Mitzenmacher 2001] shows that choosing the least loaded option among only two random destinations exponentially reduces the expected maximum load compared to a purely random choice, a result that underpins simple and scalable policies.

In systems with multiple kernels, it was necessary from the start to decide how to distribute the machine's resources among the instances. In Hive, the sharing mechanisms are implemented through cooperation between cells, but the policy is assigned to a user process, the only one with a global view of the system, responsible for deciding from which cells to allocate memory and how to schedule processes [Chapin et al. 1995]. In Disco, the monitor applies global policies over the virtual machines, moving memory between them and dynamically scheduling the vCPUs to balance the load [Bugnion et al. 1997]. Cellular Disco combines an idle balancer, which uses only local information, with a periodic balancer, which queries a global load tree [Govil et al. 2000]. In Barrelfish, a knowledge base gathers hardware information and run-time measurements, answering optimization queries, such as the allocation of drivers to cores [Baumann et al. 2009, Schüpbach et al. 2008]. In HPC-focused multikernels, management tends to be exposed as an interface. In IHK, the master kernel takes control of the physical memory mapping and offers operations to reserve cores and memory, and to initialize, query, shut down, and destroy instances, but it does not define policies about when to use them [Shimosawa et al. 2014].

Placement decisions depend on information about the state of the nodes. Mansouri-Samani and Sloman [Mansouri-Samani and Sloman 1993] define a functional model for monitoring distributed systems divided into four activities (generation, processing, dissemination, and presentation of information) and point to information staleness as the main problem of monitoring, which directly affects an orchestrator that makes decisions based on remote state. In microservices, observability is organized today around three sources: metrics, logs, and traces [Faseeha et al. 2025]. LDMS [Agelastos et al. 2014] shows that it is possible to continuously monitor an en-

tire system, even in HPC, with collector processes on the nodes and aggregator processes that gather the data, with low overhead on the applications. Systems with multiple kernels themselves also have monitoring mechanisms: Barrelfish's knowledge base incorporates measurements such as the communication latency between pairs of cores [Baumann et al. 2009], Cellular Disco's periodic balancer depends on a constantly updated load tree [Govil et al. 2000], and PIKA measures the time of each application to guide its choices [Beckmann et al. 2014]. However, this information is collected and consumed on a single machine, without sharing among the nodes of a cluster.

A major motivation for dividing the system into multiple kernels is application availability. In Hive, a hardware or software failure affects only the processes that use the resources of the affected cell. The system protects each page against writes from other cells, preemptively discards the pages that a failed cell could have corrupted, and detects failures through heuristics, whose confirmation requires consensus among the surviving cells [Chapin et al. 1995]. Disco treats each virtual machine as a unit of software fault containment [Bugnion et al. 1997], and Cellular Disco extends this containment to hardware faults: in hundreds of fault-injection experiments, recovery was successful in all cases, in less than half a second [Govil et al. 2000]. In addition to containing faults, division into multiple kernels is also used to isolate application performance. Pisces [Ouyang et al. 2015] allows the coexistence, on the same node, of multiple independent OS environments, called enclaves, each running a lightweight co-kernel that manages a partition of hardware resources, treating performance isolation as a primary requirement.

4.3. Orchestration

Current orchestrators have been enormously influenced by Google's experience with Borg, which was consolidated in Kubernetes into a declarative model, that is, the user describes the desired state, and control loops continuously reconcile the observed state with it [Burns et al. 2016]. A review of orchestrators such as Kubernetes, Docker Swarm, and Apache Mesos identifies the scarcity of proposals for resource-utilization-driven orchestration and compares the suitability of each tool [Swetha et al. 2025]. Along the same lines, a two-stage container deployment distributes containers among virtual machines with coarse granularity through a greedy algorithm, while a genetic algorithm divides the resources within each virtual machine [Lu et al. 2025], a structure similar to choosing a node and then performing local partitioning, something that will be adopted in this work.

For workloads that require stronger isolation than that of containers, the predominant approach is to integrate lightweight virtual machines into existing orchestrators. Firecracker [Agache et al. 2020] is a minimal virtual machine monitor, developed to run functions and containers from different customers in microVMs, and Kata Containers [OpenInfra Foundation 2026] runs each Kubernetes pod in a lightweight virtual machine, integrated through the container runtime interface (CRI). Orchestrated microVMs are, therefore, the comparison baseline adopted in this work.

In the multikernel context, instance management remains local. In the Hobbes project, a node manager is responsible for the lifecycle of distinct enclaves on the same node and for the shared memory mechanisms between them [Kocoloski et al. 2015]. For the Linux multikernel proposal, the kerf tool [Multikernel Technologies 2025] manages

the creation, configuration, monitoring, and destruction of instances, being the tool closest to an orchestrator, but limited to a single machine.

5. Methodology

The methodology used in this work is theoretical-practical in nature, combining analysis of orchestration algorithms and mechanisms with their respective implementations and an experimental evaluation. The approach is quantitative, based on identifying the benefits and comparing performance between applications using measurements of the behavior of the orchestrator and of instances under controlled workloads. Finally, the research level is exploratory, testing and verifying different possibilities that can be used to manage this new type of technology, given that distributed orchestration of the multikernel architecture has not yet been documented in the literature.

5.1. Characterize and define the orchestrable unit

In the first stage, the state of the art will be surveyed and the orchestrable unit will be defined, that way, the kernel version and the physical machines used in the work will be fixed. The multikernel platform will be studied experimentally regarding boot time, the way cores and memory are reserved, the instances' access to the network, and possible signals available for detecting failures. Based on this characterization, what makes up an instance (dedicated cores, memory region, image, and kernel configuration) and the desired-state model maintained by the orchestrator will be formally defined.

5.2. Design and implementation of the prototype

The second stage accomplishes the objectives of designing the architecture and implementing the lifecycle and placement. The control plane will be implemented in order to receive instance requests, described by state. The agent, running on the host kernel of each node, will be commanded by the control plane, performing resource partitioning and the initialization, stopping, and removal of instances, reporting the state of each of them. Initially, two basic placement policies will be implemented: *first-fit* and choosing the less loaded node between two randomly drawn ones [Mitzenmacher 2001].

5.3. Resource partitioning, monitoring, and recovery

In the third stage, resource partitioning and monitoring will be carried out. The orchestrator will operate in two modes: static, with predefined partitions on each node, and dynamic, with cores and memory partitioned on each request. The agent will be a periodic collector, covering the state and resource usage of each instance and of the nodes; the control plane must aggregate this information and make decisions based on it. Failures will be detected by the absence of signals from the agent and by events; recovery will consist of restarting the instance on the same node or relocating it to another one that is available and healthy.

5.4. Analysis and refinements

The fourth stage consists of carrying out the experimental evaluation, which will be conducted on a set of physical nodes connected on the same local network~~, in addition to testing the tool with specific experiments: ~~. To avoid mixing the cost of the orchestrator with the cost of the architecture, the comparison with microVMs will be performed under

equivalent conditions. Both unit types will receive the same number of dedicated cores and the same amount of memory, on the same nodes, and under the same workloads. The orchestration of the microVMs is yet to be defined. An existing orchestrator will be adopted, or a dedicated one will be built, according to which option best allows the cost of orchestration to be measured separately from the cost of the workloads. The evaluation will consist of the following experiments:

- **Research and explore workloads:** Workloads will be selected from established benchmarks, according to criteria fixed before the tests are executed, and must cover both scenarios in which the multikernel architecture tends to be favored and scenarios in which it tends to be disfavored~~. In this~~ , that way, the results are not conditioned on a choice of workloads that favors the proposal and can provide clear and meaningful results. Candidate workloads includes `stress-ng` for CPU, cache, and memory, `pgbench` as a database workload, and `iperf3` as I/O-intensive workloads.
- **Provisioning and orchestrator cost:** time between the request and the instance's response, broken down into partitioning, kernel boot, and availability, compared to the boot time of microVMs. Also analyze CPU and memory consumption of the agent and the control plane.
- **Density and interference:** run latency-sensitive, CPU-bound, and memory-bound workloads alongside other applications, comparing multikernel and microVMs.
- **Static and dynamic partitioning:** sequences of requests with many different sizes, measuring the utilization of cores and memory on each node, rejected requests, and provisioning time.
- **Failure detection and recovery:** failures will be injected by forcing kernel panics in instances, terminating agents, and disconnecting nodes, measuring the time to detect each failure, the time until the instance is running again, the recovery success rate, and the CPU and network overhead of the monitoring itself. It should be clear that the state held in the memory of the failed instance is lost, state persistence or replication is a responsibility of the application.

Each experiment will be repeated at least 20 times, with results being reported as median and 99th percentile. The hypothesis is considered confirmed if the median share of orchestration is below a fixed threshold of the total provisioning time, if the degradation of the latency is significantly smaller for multikernel instances than for microVMs, and if the dynamic partitioning yields higher core utilization and a lower rejection rate than static partitioning. With this data collected, an analysis will be performed and more specific modifications to the orchestrator may be made, in order to improve its performance.

6. Expected Results

The main expected result is a functional prototype of the K-ORCH tool, capable of provisioning, placing, and monitoring multikernel instances across multiple nodes. The proposal seeks to understand and explore the best paths for the specific characteristics of the multikernel architecture, creating ~~this robust toolfor the orchestration of distributed systems~~a robust tool, and using it as a prototype to test the hypothesis presented in Section 2.

It is also intended to create a record of the parameters used and of the greatest difficulties in carrying out orchestration, mapping and documenting algorithms, main ideas, performance, and other metrics. In this way, it will also be possible to offer a characterization of the costs and limits of distributed multikernel orchestration. Finally, the code, documentation, and experimental scripts will be publicly released and the intention is to disseminate the proposal and the results obtained through a paper submitted to a relevant conference or journal in this research area.

7. Schedule

Table 1 presents the schedule organized into 8 quarters. Quarters one and two focus on studying the multikernel platform and formally defining the orchestrable unit used, creating a reproducible experimental environment. Quarters three and four will be used to design the general operation of the orchestrator, and then implement the orchestration agents in order to carry out the application lifecycle and placement, with a functional orchestration unit as a milestone at the end of the year, serving as a proof of concept. Quarters five and six will be dedicated to resource partitioning, monitoring, and instance recovery, as well as preparing the environment for comparison with microVMs. Finally, in quarters seven and eight, tests and analyses will be carried out regarding the performance of the orchestration and of the chosen applications, that is, an experimental evaluation focused on refining the orchestrator and consolidating the results.

Table 1. Task schedule organized by quarter.

Activity	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8
Survey of the state of the art	•	•						
Experimental characterization of the multikernel platform	•	•						
Definition of the orchestrable unit and state model		•						
Architecture design			•					
Implementation of lifecycle and placement			•	•				
Writing, delivery, and defense of the TCC			•	•				
Implementation of resource partitioning					•	•		
Implementation of instance monitoring and recovery					•	•		
Preparation of the evaluation environment and reference microVMs					•	•		
Experimental and comparative evaluation							•	•
Orchestrator refinement							•	•
Writing ~~of papers and the final text~~ and submission of papers				•		•	•	•
Master's defense								•

8. ~~Final Remarks~~

~~This work proposes K-ORCH, an orchestrator that extends the management of multikernel instances, currently restricted to a single machine, to environments with multiple nodes. The proposal combines the model of current orchestrators with the partitioning mechanisms specific to the multikernel architecture and will be evaluated against orchestrated microVMs in terms of provisioning cost, density, and performance interference between instances. The expected contribution is to provide a third way for environments that need the best of containers and virtual machines, providing enough abstraction to scale this architecture to large production systems with ease and reliability. In addition, it is expected to characterize the costs and limits of distributed orchestration of the multikernel architecture.~~

References

Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 419–434, Santa Clara - CA, USA.

Agelastos, A., Allan, B., Brandt, J., Cassella, P., Enos, J., Fullop, J., Gentile, A., Monk, S., Naksinehaboon, N., Ogden, J., Rajan, M., Showerman, M., Stevenson, J., Taerat, N., and Tucker, T. (2014). The lightweight distributed metric service: A scalable infrastructure for continuous monitoring of large scale computing systems and applications. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 154–165, New Orleans - LA, USA.

AMD (2024). AMD launches 5th gen AMD EPYC CPUs, maintaining leadership performance and features for the modern data center. `https://www.amd.com/en/newsroom/press-releases/2024-10-10-amd-launches-5th-gen-amd-epyc-cpus-maintaining-le.html`. Acessado em 01/11/2026.

Baumann, A., Barham, P., Dagand, P.-E., Harris, T., Isaacs, R., Peter, S., Roscoe, T., Schüpbach, A., and Singhania, A. (2009). The multikernel: A new OS architecture for scalable multicore systems. In *22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 29–44, Big Sky - MT, USA.

Beckmann, N. Z., Gruenwald III, C., Johnson, C. R., Kasture, H., Sironi, F., Agarwal, A., Kaashoek, M. F., and Zeldovich, N. (2014). PIKA: A network service for multikernel operating systems. Technical Report MIT-CSAIL-TR-2014-002, MIT Computer Science and Artificial Intelligence Laboratory, Cambridge - MA, USA.

Boyd-Wickizer, S., Chen, H., Chen, R., Mao, Y., Kaashoek, F., Morris, R., Pesterev, A., Stein, L., Wu, M., Dai, Y., Zhang, Y., and Zhang, Z. (2008). Corey: An operating system for many cores. In *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 43–57, San Diego - CA, USA.

Boyd-Wickizer, S., Clements, A. T., Mao, Y., Pesterev, A., Kaashoek, M. F., Morris, R., and Zeldovich, N. (2010). An analysis of Linux scalability to many cores. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Vancouver - BC, Canada.

Bugnion, E., Devine, S., Govil, K., and Rosenblum, M. (1997). Disco: Running commodity operating systems on scalable multiprocessors. *ACM Transactions on Computer Systems*, 15(4):412–447.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue*, 14(1).

Casavant, T. L. and Kuhl, J. G. (1988). A taxonomy of scheduling in general-purpose distributed computing systems. *IEEE Transactions on Software Engineering*, 14(2):141–154.

Chapin, J., Rosenblum, M., Devine, S., Lahiri, T., Teodosiu, D., and Gupta, A. (1995). Hive: Fault containment for shared-memory multiprocessors. In *15th ACM Symposium on Operating Systems Principles (SOSP)*, pages 12–25, Copper Mountain - CO, USA.

Faseeha, U., Syed, H. J., Samad, F., Zehra, S., and Ahmed, H. (2025). Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms. *IEEE Access*, 13:72011–72039.

Gerofi, B., Ishikawa, Y., Riesen, R., Wisniewski, R. W., Park, Y., and Rosenburg, B. (2016a). A multi-kernel survey for high-performance computing. In *6th International Workshop on Runtime and Operating Systems for Supercomputers (ROSS)*, Kyoto - Japan.

Gerofi, B., Takagi, M., Hori, A., Nakamura, G., Shirasawa, T., and Ishikawa, Y. (2016b). On the scalability, performance isolation and device driver transparency of the IHK/McKernel hybrid lightweight kernel. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, Chicago - IL, USA.

Govil, K., Teodosiu, D., Huang, Y., and Rosenblum, M. (2000). Cellular Disco: Resource management using virtual clusters on shared-memory multiprocessors. *ACM Transactions on Computer Systems*, 18(3):229–262.

Kocoloski, B., Lange, J., Abbasi, H., Bernholdt, D. E., Jones, T. R., Dayal, J., Evans, N., Lang, M., Lofstead, J., Pedretti, K., and Bridges, P. G. (2015). System-level support for composition of applications. In *5th International Workshop on Runtime and Operating Systems for Supercomputers (ROSS)*, Portland - OR, USA.

Lo, D., Cheng, L., Govindaraju, R., Ranganathan, P., and Kozyrakis, C. (2015). Heracles: Improving resource efficiency at scale. In *42nd International Symposium on Computer Architecture (ISCA)*, pages 450–462, Portland - OR, USA.

Lu, C., Zhou, J., and Zou, Q. (2025). An optimized approach for container deployment driven by a two-stage load balancing mechanism. *PLOS ONE*, 20(1). Artigo e0317039.

Mansouri-Samani, M. and Sloman, M. (1993). Monitoring distributed systems. *IEEE Network*, 7(6):20–30.

Mitzenmacher, M. (2001). The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104.

Multikernel Technologies (2025). kerf: Multikernel management system. `https://github.com/multikernel/kerf`. Acessado em 10/10/2026.

NIST (2019). CVE-2019-5736: runc container breakout. https://nvd.nist.gov/vuln/detail/CVE-2019-5736. National Vulnerability Database. Acessado em 23/10/2026.

NIST (2024). CVE-2024-21626: runc container breakout. https://nvd.nist.gov/vuln/detail/CVE-2024-21626. National Vulnerability Database. Acessado em 23/10/2026.

OpenInfra Foundation (2026). Kata Containers: The speed of containers, the security of VMs. https://katacontainers.io. Acessado em 23/10/2026.

Ouyang, J., Kocoloski, B., Lange, J. R., and Pedretti, K. (2015). Achieving performance isolation with lightweight co-kernels. In *24th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, pages 149–160, Portland - OR, USA.

Park, Y., Van Hensbergen, E., Hillenbrand, M., Inglett, T., Rosenburg, B., Ryu, K. D., and Wisniewski, R. W. (2012). FusedOS: Fusing LWK performance with FWK functionality in a heterogeneous environment. In *24th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pages 211–218, New York - NY, USA.

Schüpbach, A., Peter, S., Baumann, A., Roscoe, T., Barham, P., Harris, T., and Isaacs, R. (2008). Embracing diversity in the Barrelfish manycore operating system. In *Workshop on Managed Many-Core Systems (MMCS)*, Boston - MA, USA.

Shimosawa, T., Gerofi, B., Takagi, M., Nakamura, G., Shirasawa, T., Saeki, Y., Shimizu, M., Hori, A., and Ishikawa, Y. (2014). Interface for heterogeneous kernels: A framework to enable hybrid OS designs targeting high performance computing on many-core architectures. In *21st International Conference on High Performance Computing (HiPC)*, pages 1–10, Goa - India.

Shivaratri, N. G., Krueger, P., and Singhal, M. (1992). Load distributing for locally distributed systems. *Computer*, 25(12):33–44.

Swetha, R., Thriveni, J., and Venugopal, K. R. (2025). Resource utilization-based container orchestration: Closing the gap for enhanced cloud application performance. *SN Computer Science*, 6. Artigo 191.

Wang, C. (2025a). Multikernel: Kernel-to-kernel isolation with elastic resource management. https://lpc.events/event/19/contributions/2074/. Linux Plumbers Conference. Acessado em 10/10/2026.

Wang, C. (2025b). [RFC patch 0/7] kernel: Introduce multikernel architecture support. https://lore.kernel.org/lkml/20250918222607.186488-1-xiyou.wangcong@gmail.com/. Linux Kernel Mailing List. Acessado em 10/10/2026.

Wentzlaff, D. and Agarwal, A. (2009). Factored operating systems (fos): The case for a scalable operating system for multicores. *ACM SIGOPS Operating Systems Review*, 43(2):76–85.